

Auditoria do Sistema + Plano de Implementação

LangNet Interface

Estado de cada etapa · Interface ↔ Código · Fluxo dos agentes
Plano das funções faltantes + testes — 22/07/2026

Auditoria do Sistema + Plano de Implementação — LangNet Interface

Data: 22/07/2026 · **Base:** código real (backend, frontend e o app efetivamente gerado pela Quântica) **Escopo:** (1) auditoria de cada etapa do pipeline, com foco na conexão **Interface ↔ Código** e no **fluxo dos agentes**; (2) plano detalhado para implementar as funções faltantes (Parte B da especificação de casos de uso), com sugestões de teste.

PARTE 1 — AUDITORIA DO SISTEMA

1.1 Sumário executivo

A auditoria trouxe uma **boa surpresa**: o pipeline de geração está **mais completo do que se supunha**. A partir do código realmente gerado para a Quântica (sessão `af4708e5`, **55 arquivos**), confirmou-se que a Geração de Código produz um **aplicativo full-stack coerente e executável**, não apenas o executor da Rede de Petri:

- **Frontend (Cara A):** 18 telas de negócio React (`frontend/src/screens/*.jsx`), uma por caso de uso, agrupadas por módulo, cada uma disparando a tarefa do agente via WebSocket.
- **Servidor de execução (Cara B + agentes):** `ws-server/` com executor real — **determinístico primeiro** (SQL direto nas tabelas do Modelo de Dados) **ou CrewAI** (`crew.kickoff()`).
- **Infra:** `backend/`, `docker-compose.yml`, Dockerfiles, `petri_net.json`, `agents.yaml`, `tasks.yaml`.

Ou seja: **a conexão Interface → Código EXISTE e funciona**, e **os agentes são realmente executados** no código gerado. O gargalo não é a geração — é a **validação em tempo de execução** do app gerado e a **ausência das funções de configuração/operação** (todas mock).

⚠ **Correção de um mal-entendido comum:** os arquivos em `framework/` (ex.: `websocket_api_tropical.py`) são o **framework legado** (`visualtasksexec`) e **não** representam o que o code gen produz hoje. A auditoria da execução deve olhar o **código gerado** (`ws-server/...`), que é auto-contido e real.

1.2 Estado de cada etapa do pipeline

#	Etapa	Estado	Output é consumido adiante?	Gaps
0	Documentos → Requisitos	✔ Real	Sim → Especificação	Nenhum crítico
1	Especificação Funcional	✔ Real	Sim → 4 consumidores	Página não auto-carrega última sessão
2	Modelo de Dados	✔ Real	Sim → Interface, adapters SQL	—
3	Interface & Protótipo (<code>ui_spec</code>)	✔ Real	Sim → telas React no código	Casamento tela → task é heurístico
4	Casos de Teste (CEG)	✔ Real	Terminal (validação)	Não realimenta o código ainda
5	Agent-Task Spec	✔ Real	Sim → <code>agents.yaml</code> , <code>tasks.yaml</code>	—
6	<code>agents.yaml</code> / <code>tasks.yaml</code>	✔ Real	Sim → Petri e Código	Tools nem sempre reais nos agentes
7	Sequência de Tarefas	✔ Real	Sim → Petri e Código	Página não auto-carrega; custo de contexto
8	Rede de Petri	✔ Real	Sim → orquestração no código	<code>task_name</code> vazio nos lugares (vínculo na lógica)
9	Geração de Código	✔ Real	Produto final (app full-stack)	Nunca validado em runtime

Leitura: as 10 etapas estão implementadas e **encadeadas** (proveniência já garantida na sessão anterior). O que falta não é "gerar" — é **rodar e validar** o que se gera, e **cobrir as áreas mock**.

1.3 Deep-dive A — Conexão Interface → Código (CONFIRMADA)

Como funciona (verificado no código gerado): 1. A etapa de Interface produz o `ui_spec` (JSON: telas, componentes com `bindTo` campo → coluna, ações). 2. Na Geração de Código, `_generate_business_screens()` (`langnetagents.py:~4132`) transforma o `ui_spec` em componentes React **por template determinístico** (o `ui_spec` **não** vai ao prompt do LLM — `code_generation.py` o carrega e versiona, e a montagem das telas é 100% Python). 3. Cada tela gera um `.jsx` de um dos 4 tipos: **form / agent / report / crud**. 4. `App.jsx` é substituído por um shell com as telas agrupadas por módulo + aba "Admin" (executor de Petri). 5. Cada tela chama `runTask(task_name, payload)` → `wsClient.js` → WebSocket `:5002` → `ws-server`.

Evidência (Quântica): 18 telas geradas (CadastrarPersonaAlvo, CalendarioMensal, GerarConteúdo...), `App.jsx` com registro de telas por módulo (Cadastros, Conteúdo, Publicação, Engajamento, Relatórios, Integrações), `wsClient.js` com o protocolo `{type:"execute_task", data:{task_name, input_data}}`.

Veredito: conexão **funcional (~90%)**. Gaps: (a) o nome da task na tela é casado por **similaridade de tokens** com a task real — pode errar; (b) telas geradas são bons protótipos, mas faltam validação de formulário mais rica, acessibilidade e testes visuais para "produção".

1.4 Deep-dive B — Fluxo dos agentes e execução (CONFIRMADO real no código gerado)

Cadeia de geração (toda real): Agent-Task Spec (markdown de agentes/tarefas) → `agents.yaml` + `tasks.yaml` (LLM, com validação de SQL nas tasks que persistem) → Rede de Petri (lugares com lógica WebSocket) → Código.

Execução no app gerado (`ws-server/websocket_server.py`): - Recebe `execute_task` via WebSocket. - **Deterministic-first:** se existe `<task>_deterministic` em `adapters.py`, roda **SQL direto** (ex.: `INSERT INTO personas/canais/problemas` — as tabelas do Modelo de Dados). Sem LLM. Rápido e confiável. - **Senão:** monta `Agent` + `Task` CrewAI a partir de `agents.yaml` / `tasks.yaml` e executa `result = crew.kickoff()` (linha ~225). Aplica `input_func` / `output_func` dos adapters.

Veredito: os agentes **são executáveis de verdade** no código gerado — tanto o caminho determinístico (CRUD → banco) quanto o caminho CrewAI (tarefas cognitivas). O que **não** foi comprovado é o **funcionamento ponta-a-ponta em runtime** (subir o ws-server + frontend e disparar tarefas reais).

Telas de agentes no LangNet (não confundir com o app gerado): - `AgentTaskPage` (Agentes & Tarefas) —  real (gera o Agent-Task Spec). - `AgentsPage`, `AgentDesignerPage`, `AgentChatPage` —  mock (dados fixos, sem backend).

1.5 Gaps reais, priorizados

Prio	Gap	Área	Impacto
● P0	App gerado nunca validado em runtime (build do frontend, subir ws-server, disparar 1 task real E2E)	Código/Agentes	Não se sabe se o produto final roda
● P0	Funções de Configuração (banco, LLM, integrações) são mock — config só via <code>.env</code>	Config	Usuário não configura pela UI
● P1	Casamento tela → task por similaridade de tokens pode errar	Interface ↔ Código	Botão pode chamar task errada
● P1	Tools dos agentes nem sempre reais (tasks "agent" dependem disso; CRUD é sólido)	Agentes	Tarefas cognitivas podem falhar
● P1	MCP (config global, descoberta, sync) todo mock	Config/Integração	Sem integração MCP real
● P2	<code>task_name</code> vazio nos lugares da Petri (vínculo só na lógica WS)	Petri	Fragilidade de manutenção
● P2	Páginas que não auto-carregam a última sessão (Especificação, Sequência)	UX	Confunde o usuário
● P2	Deploy, Monitoramento, Interação (5 telas) mock	Operação	Sem operação real
● P3	Casos de Teste não realimentam a geração (testes não viram testes no código)	Qualidade	Oportunidade

PARTE 2 — PLANO DE IMPLEMENTAÇÃO

2.1 Estratégia e priorização

O plano ataca duas frentes em paralelo: - **Frente A — Fechar o núcleo (validar o que já se gera):** provar que o app gerado roda (P0 técnico). - **Frente B — Implementar as funções faltantes (Parte B dos casos de uso):** começando pela **Configuração** (a que o usuário mais sente falta), depois MCP, Operação e Interação.

Ordem sugerida (fases): **F0** → **F1** → **F2** → **F3** → **F4** → **F5** (detalhadas em 2.2–2.7). Cada fase termina com **backup + commit + push + testes** (padrão já adotado no projeto).

2.2 F0 — Validar o app gerado em runtime (Frente A, P0) · esforço: M

Objetivo: provar que o código gerado da Quântica **roda de ponta a ponta**. **Tarefas:** 1. Extrair o ZIP do `af4708e5`; `docker-compose up` (ou rodar `ws-server` + `frontend` manualmente). 2. Subir o `ws-server` (porta 5002) e o frontend; abrir uma tela (ex.: Cadastrar Pessoa). 3. Disparar uma **task determinística** (CRUD) → conferir `INSERT` real no banco do app. 4. Disparar uma **task CrewAI** (cognitiva) → conferir `crew.kickoff()` retornando resultado. 5. Corrigir o que travar (imports, env, task matching) **no gerador** (não no artefato).

Testes de verificação: - **T-F0.1** `docker-compose up` sobe os 3 serviços sem erro. - **T-F0.2** POST WS `execute_task` `cadastrar_persona_alvo` → linha nova em `personas` (assert no banco). - **T-F0.3** Uma task "agent" retorna `task_completed` com `result` não-vazio. - **T-F0.4** `npm run build` do frontend gerado conclui sem erro.

2.3 F1 — Configurações do Sistema reais (UC-P01, Frente B, P0) · esforço: M/G

Objetivo: tornar a tela **Configurações** funcional: banco de dados, provedor LLM e integrações, com persistência e validação. (Hoje tudo em `backend/.env` e a tela é mock.)

Backend: - Novo router `app/routers/settings.py`: `GET /api/settings`, `PUT /api/settings`, `POST /api/settings/test-db`, `POST /api/settings/test-llm`. - Tabela `system_settings` (chave/valor por seção; segredos cifrados). Ou arquivo de config gerenciado + reload. **Segredos nunca retornam em claro** (RN-P01.1). - `test-db`: tenta conectar com as credenciais informadas e retorna status real (nº de tabelas, tamanho). - `test-llm`: consulta `/v1/models` do endpoint informado (LM Studio/DeepSeek/OpenAI) e valida o modelo.

Frontend: - Ligar `SettingsPage` às novas APIs (abas Geral / LLMs / **Banco de Dados** / Integrações). - Botões *Testar conexão* (banco e LLM) com feedback real; *Salvar* só habilita após teste OK (RN-P01.2).

Testes: - **T-F1.1** *Testar conexão* com credenciais corretas → "conectado, 19 tabelas". - **T-F1.2** *Testar conexão* com senha errada → erro do driver, sem salvar. - **T-F1.3** Salvar provedor LLM → nova geração passa a usar o modelo salvo (sem editar `.env`). - **T-F1.4** `GET /api/settings` nunca retorna senha/API key em claro.

2.4 F2 — MCP real (UC-P02/P03/P04, Frente B, P1) · esforço: G

Objetivo: transformar as 3 telas MCP (mock) em integração real.

Backend: - Router `app/routers/mcp.py` + tabelas `mcp_servers`, `mcp_services`, `mcp_sync_jobs`. - `POST /mcp/servers` (registrar), `POST /mcp/servers/{id}/test` (handshake real → capacidades), `GET /mcp/services` (descoberta a partir dos servidores ativos), `POST /mcp/sync` (+ resolução de conflito). - Um servidor só fica "ativo" após teste OK (RN-P02.1); credenciais protegidas.

Frontend: ligar `McpGlobalConfigPage`, `McpServiceDiscoveryPage`, `McpStateSyncPage` às APIs.

Testes: - **T-F2.1** Registrar servidor + testar → status "ativo" e capacidades listadas. - **T-F2.2** Descoberta só lista serviços de servidores ativos. - **T-F2.3** Sync com conflito → escolha local/remoto/merge aplicada e auditada.

2.5 F3 — Robustez do núcleo (P1) · esforço: M

Objetivo: endurecer os pontos frágeis achados na auditoria. - **Casamento tela → task determinístico:** gravar no `ui_spec` o `task_name` canônico escolhido na etapa de Interface (em vez de recasar por similaridade no code gen). Teste: **T-F3.1** toda ação de tela aponta para uma task existente em `tasks.yaml` (assert 100%). - **Tools dos agentes:** validar, na geração de `agents.yaml`, que agentes de tasks "agent" têm tools reais; senão, marcar aviso. Teste: **T-F3.2** relatório de agentes sem tools. - `task_name` **nos lugares da Petri:** preencher o campo além da lógica. Teste: **T-F3.3** todo lugar com lógica WS tem `task_name` coerente.

2.6 F4 — Operação: Deploy + Monitoramento (UC-P05/P06, P2) · esforço: G

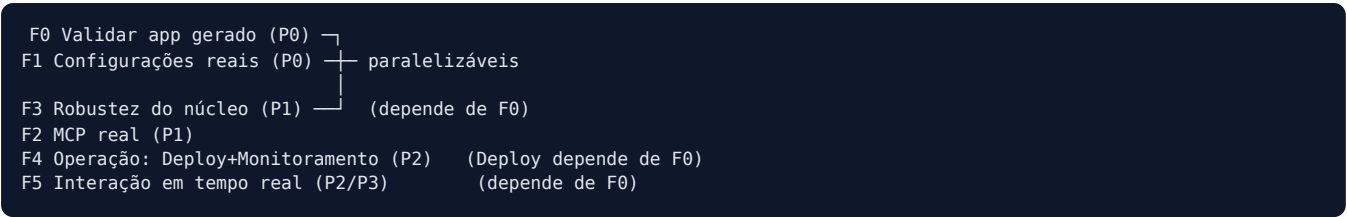
- **Deploy:** router que empacota o app gerado e publica (Docker/local); pré-requisito = geração concluída. Teste: **T-F4.1** deploy local sobe o app; **T-F4.2** deploy bloqueado sem código gerado.
- **Monitoramento:** mover a config do Langfuse para as Configurações (UC-P01) e persistir; consumir traces. Teste: **T-F4.3** conectar Langfuse via config salva e listar traces.

2.7 F5 — Interação em tempo real (UC-P07, P2/P3) · esforço: G

Tornar reais as 5 telas de Interação, conectando-as ao **app gerado em execução** (WebSocket :5002): Chat com Agentes, Estado do Sistema, Gestão de Artefatos, Formulários Dinâmicos, Designer. Dependem de **F0** (app rodando). Teste: **T-F5.1** Chat envia mensagem a um agente vivo

e recebe resposta.

2.8 Roadmap sugerido



Sugestão de ordem prática: F0 e F1 primeiro (destravam validação e configuração), depois F3 (corrige o que F0 revelar), então F2, F4 e F5.

2.9 Estratégia de testes (transversal)

Nível	O que	Ferramenta
Unit (backend)	routers novos (settings, mcp): validação, cifragem de segredos, test-db/test-llm	pytest
Integração	geração → app roda → task real grava no banco (F0)	script E2E + assert no MySQL
Contrato	<code>GET /settings</code> nunca vaza segredo; ação de tela → task existente	pytest + checagem de schema
E2E UI	fluxos das telas de config (testar conexão, salvar)	Playwright (já usado no projeto)
Regressão	rastreabilidade continua gravando proveniência a cada geração	script de verificação (já existe)

Princípio (do projeto): correções vêm de **features do LangNet**, nunca de edição manual dos artefatos da Quântica; cada fase com **backup + commit + push + teste** antes de avançar.

Anexos

- Especificação de casos de uso: <docs/spec-langnet/especificacao-casos-de-uso.pdf>
- Rastreabilidade entre artefatos: <docs/validacao-quantica/validacao-quantica.pdf>
- Manual do usuário do pipeline: <docs/validacao-quantica/manual-langnet.pdf>